Day 15



cademia.ai

Text Clustering and Topic Modeling

Academia.ai © 2026 | Text Clustering and Topic Modeling

- Although supervised techniques, such as classification, have reigned supreme over the last few years in the industry, the potential of unsupervised techniques such as text clustering cannot be understated. Text clustering aims to group similar texts based on their semantic content, meaning, and relationships.
- As illustrated in Figure 5-1, the resulting clusters of semantically similar documents not only facilitate efficient categorization of large volumes of unstructured text but also allow for quick exploratory data analysis.

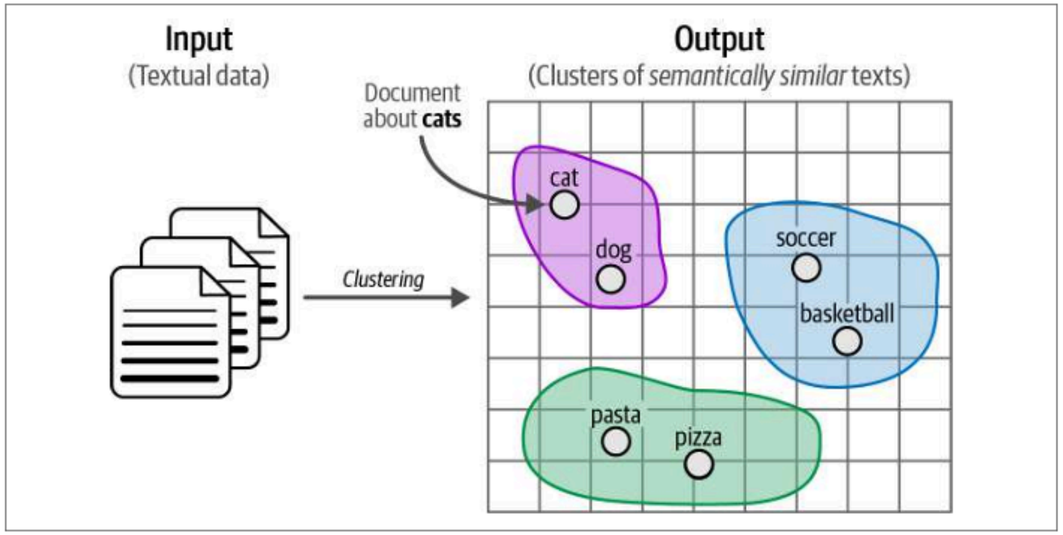


Figure 5-1. Clustering unstructured textual data.

- The recent evolution of language models, which enable contextual and semantic representations of text, has enhanced the effectiveness of text clustering. Language is more than a bag of words, and recent language models have proved to be quite capable of capturing that notion. Text clustering, unbound by supervision, allows for creative solutions and diverse applications, such as finding outliers, speedup labeling, and finding incorrectly labeled data.
- Text clustering has also found itself in the realm of topic modeling, where we want to discover (abstract) topics that appear in large collections of textual data. As shown in Figure 5-2, we generally describe a topic using keywords or keyphrases and, ideally, have a single overarching label.
-

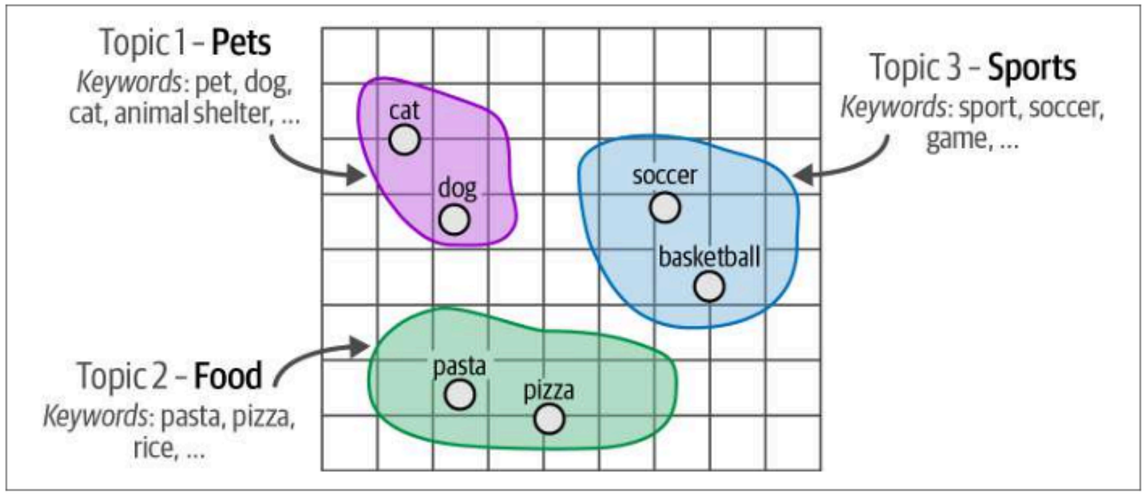


Figure 5-2. Topic modeling is a way to give meaning to clusters of textual documents.

A Common Pipeline for Text Clustering

- Text clustering allows for discovering patterns in data that you may or may not be familiar with. It allows for getting an intuitive understanding of the task, for example, a classification task, but also of its complexity. As a result, text clustering can become more than just a quick method for exploratory data analysis. Although there are many methods for text clustering, from graph-based neural net works to centroid-based clustering techniques, a common pipeline that has gained popularity involves three steps and algorithms:
- 1. Convert the input documents to embeddings with an embedding model.
- 2. Reduce the dimensionality of embeddings with a dimensionality reduction model.
- 3. Find groups of semantically similar documents with a cluster model.

Embedding Documents

- The first step is to convert our textual data to embeddings, as illustrated in Figure 5-3. Recall from previous chapters that embeddings are numerical representations of text that attempt to capture its meaning.

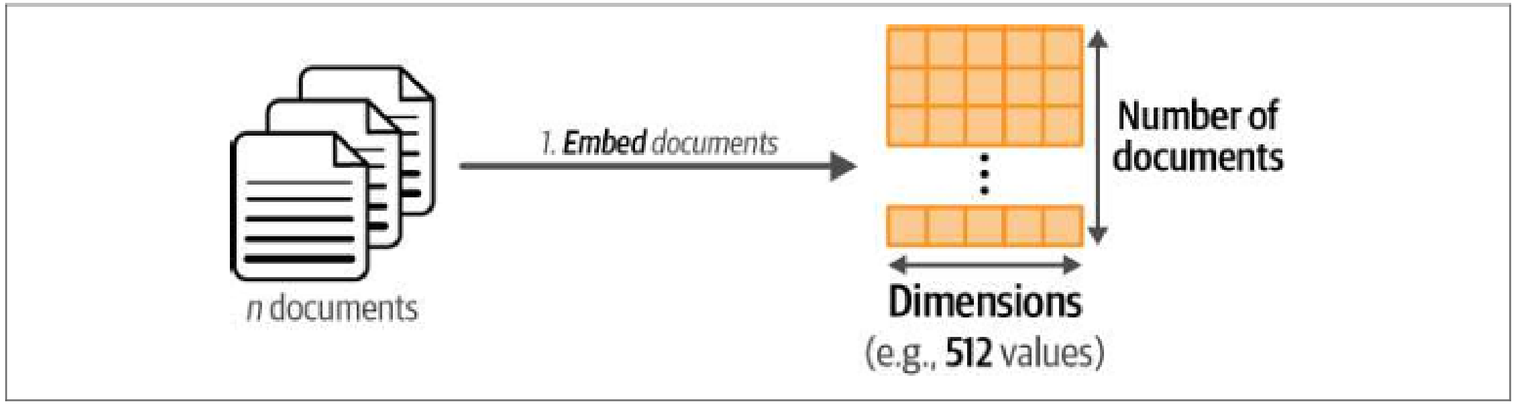


Figure 5-3. Step 1: We convert documents to embeddings using an embedding model.

- Choosing embedding models optimized for semantic similarity tasks is especially important for clustering as we attempt to find groups of semantically similar documents. Fortunately, most embedding models at the time of writing focus on just that, semantic similarity

Reducing the Dimensionality of Embeddings

- Before we cluster the embeddings, we will first need to take their high dimensionality into account. As the number of dimensions increases, there is an exponential growth in the number of possible values within each dimension. Finding all subspaces within each dimension becomes increasingly complex. As a result, high-dimensional data can be troublesome for many clustering tech niques as it gets more difficult to identify meaningful clusters. Instead, we can make use of dimensionality reduction.
- As illustrated in Figure 5-4, this technique allows us to reduce the size of the dimensional space and represent the same data with fewer dimensions. Dimensionality reduction techniques aim to preserve the global structure of high-dimensional data by finding low-dimensional representations.

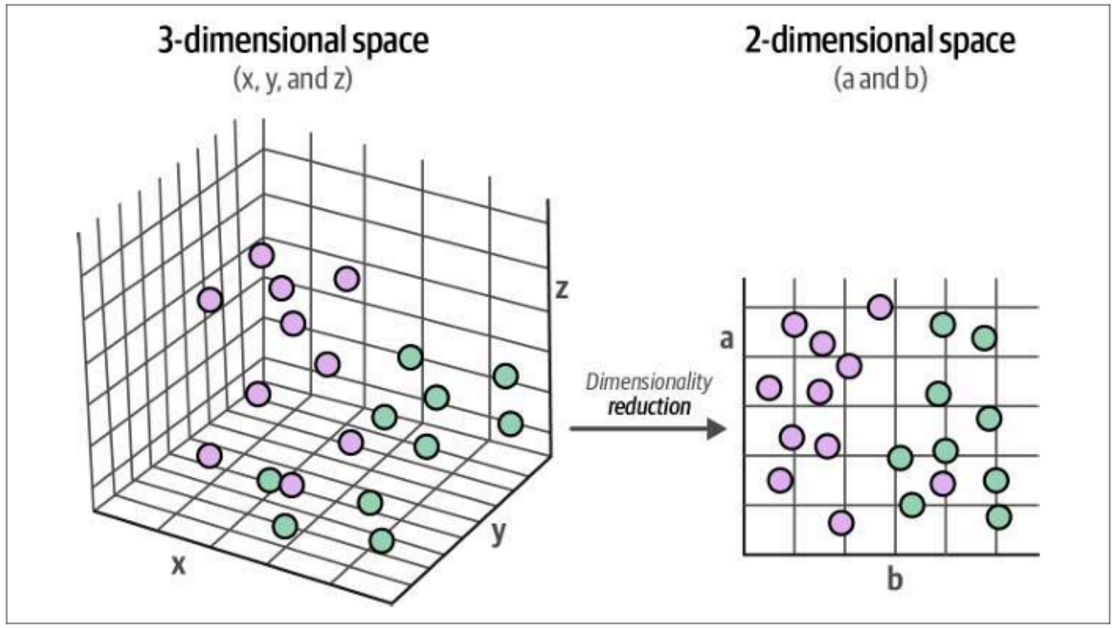


Figure 5-4. Dimensionality reduction allows data in high-dimensional space to be compressed to a lower-dimensional representation.

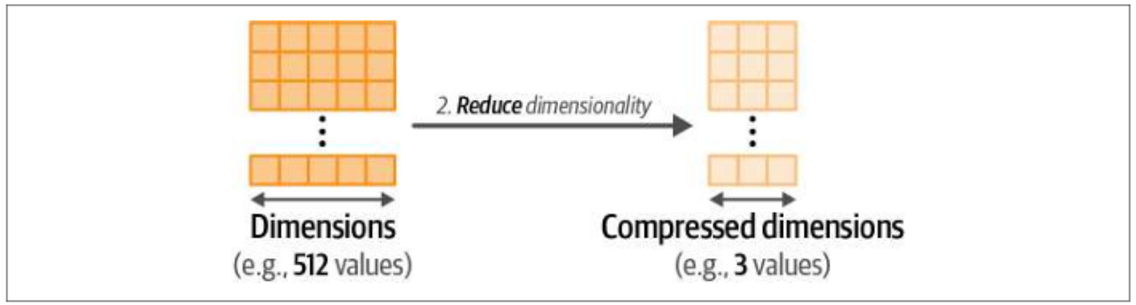


Figure 5-5. Step 2: The embeddings are reduced to a lower-dimensional space using dimensionality reduction.

- Note that this is a compression technique and that the underlying algorithm is not arbitrarily removing dimensions. To help the cluster model create meaningful clus ters, the second step in our clustering pipeline is therefore dimensionality reduction, as shown in Figure 5-5.

Cluster the Reduced Embeddings

The third step is to cluster the reduced embeddings, as illustrated in Figure 5-6

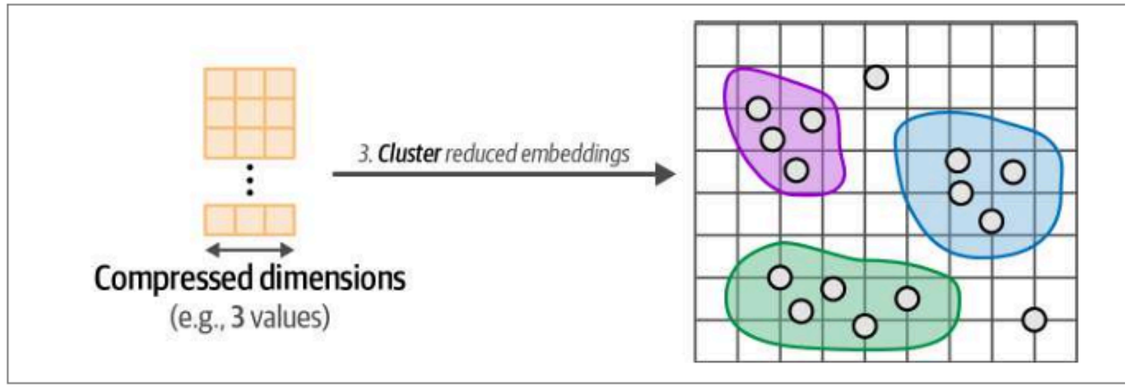


Figure 5-6. Step 3: We cluster the documents using the embeddings with reduced dimensionality.

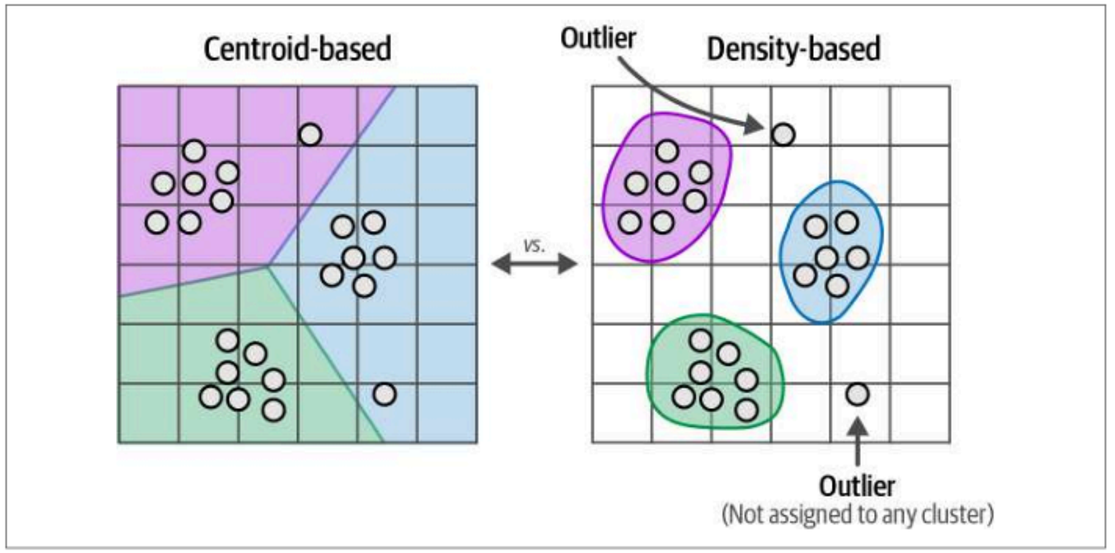


Figure 5-7. The clustering algorithm not only impacts how clusters are generated but also how they are viewed.

Although a common choice is a centroid-based algorithm like k-means, which requires a set of clusters to be generated, we do not know the number of clusters beforehand. Instead, a density-based algorithm freely calculates the number of clus ters and does not force all data points to be part of a cluster, as illustrated in Figure 5-7.

From Text Clustering to Topic Modeling

- Text clustering is a powerful tool for finding structure among large collections of documents. In our previous example, we could manually inspect each cluster and identify them based on their collection of documents. For instance, we explored a cluster that contained documents about sign language. We could say that the topic of that cluster is “sign language.”
- This idea of finding themes or latent topics in a collection of textual data is often referred to as topic modeling. Traditionally, it involves finding a set of keywords or phrases that best represent and capture the meaning of the topic, as we illustrate in Figure 5-9

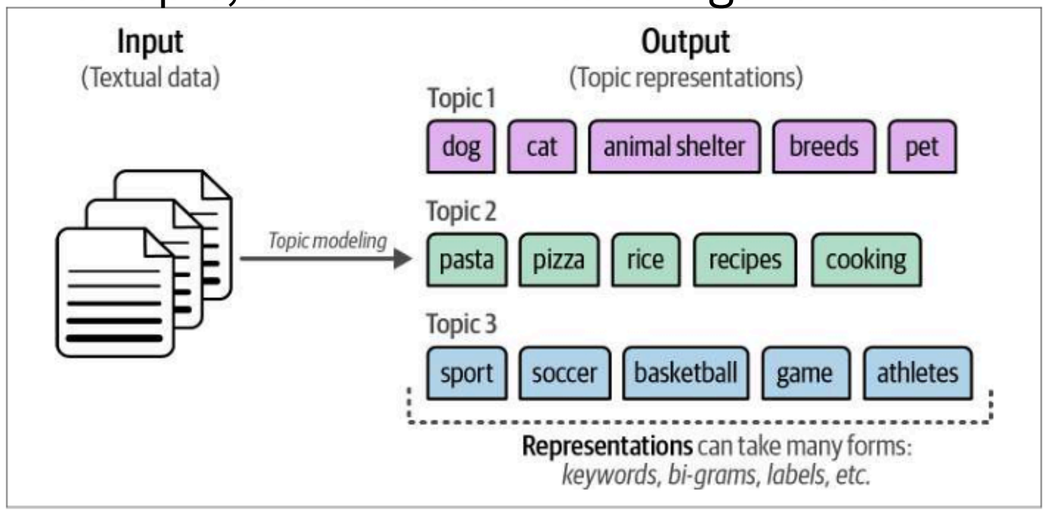


Figure 5-9. Traditionally, topics are represented by a number of keywords but can take other forms.

- Instead of labeling a topic as “sign language,” these techniques use keywords such as “sign,” “language,” and “translation” to describe the topic. As such, this does not give a single label to a topic and instead requires the user to understand the meaning of the topic through those keywords.
- Classic approaches, like latent Dirichlet allocation, assume that each topic is charac terized by a probability distribution of words in a corpus’s vocabulary.5 Figure 5-10 demonstrates how each word in a vocabulary is scored against its relevance to each topic.

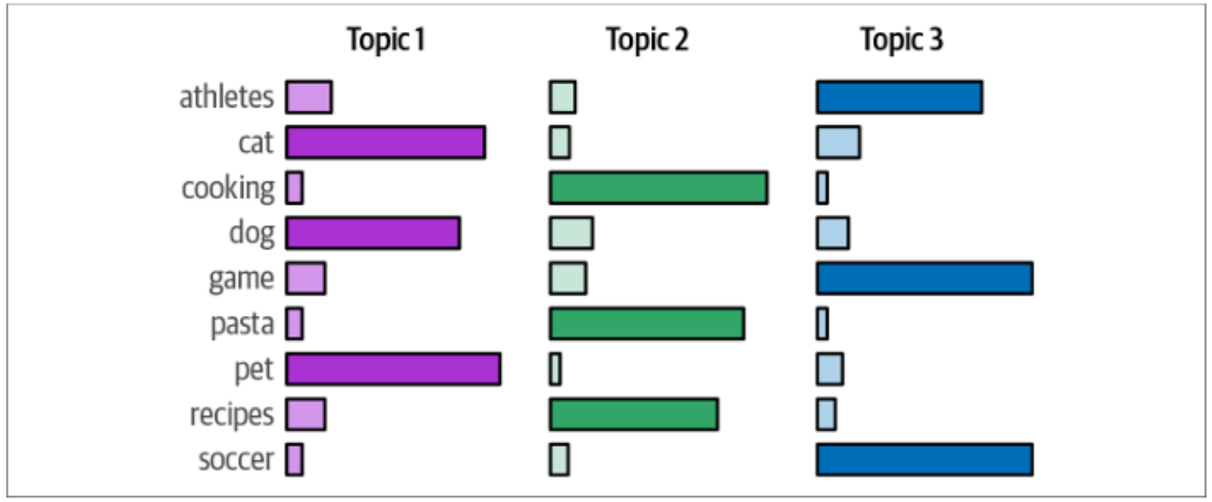


Figure 5-10. Keywords are extracted based on their distribution over a single topic.



- These approaches generally use a bag-of-words technique for the main features of the textual data, which does not take the context nor the meaning of words and phrases into account. In contrast, our text clustering example does take both into account as it relies on Transformer-based embeddings that are optimized for semantic similarity and contextual meaning through attention.
- In this section, we will extend text clustering into the realm of topic modeling through a highly modular text clustering and topic modeling framework, namely BERTopic.

BERTopic: A Modular Topic Modeling Framework

- BERTopic is a topic modeling technique that leverages clusters of semantically similar texts to extract various types of topic representations.6 The underlying algorithm can be thought of in two steps.
- First, as illustrated in Figure 5-11, we follow the same procedure as we did before in our text clustering example. We embed documents, reduce their dimensionality, and finally cluster the reduced embedding to create groups of semantically similar documents.

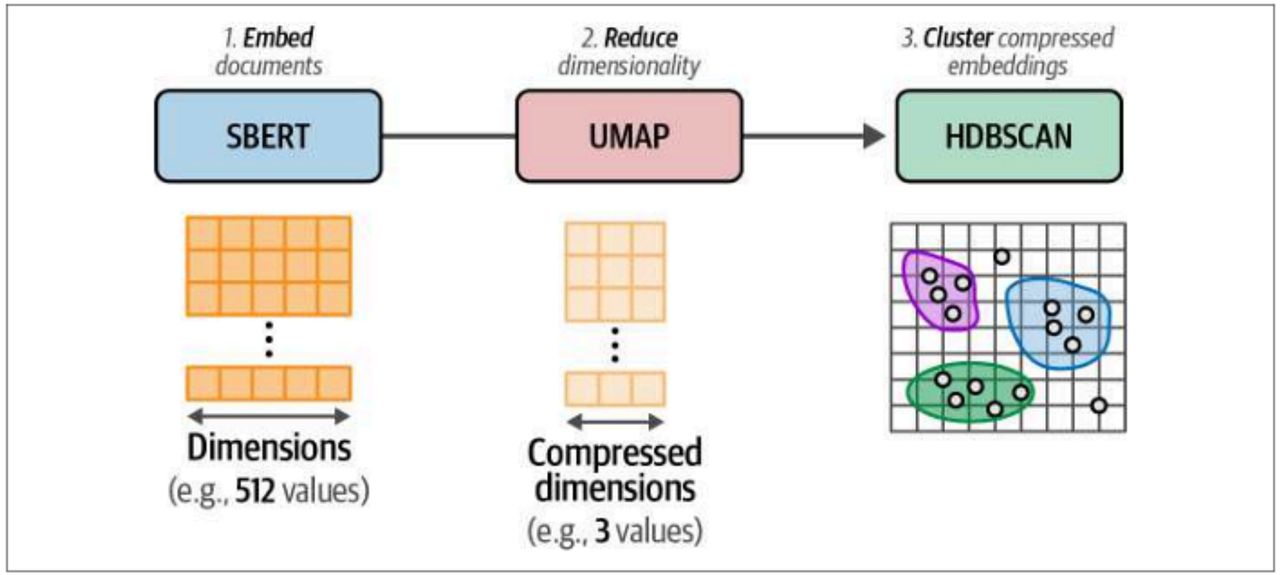


Figure 5-11. The first part of BERTopic's pipeline is to create clusters of semantically similar documents.

- Second, it models a distribution over words in the corpus's vocabulary by leveraging a classic method, namely bag-of-words. The bag-of-words, as we discussed briefly in Chapter 1 and illustrate in Figure 5-12, does exactly what its name implies, counting the number of times each word appears in a document. The resulting representation could be used to extract the most frequent words inside a document.

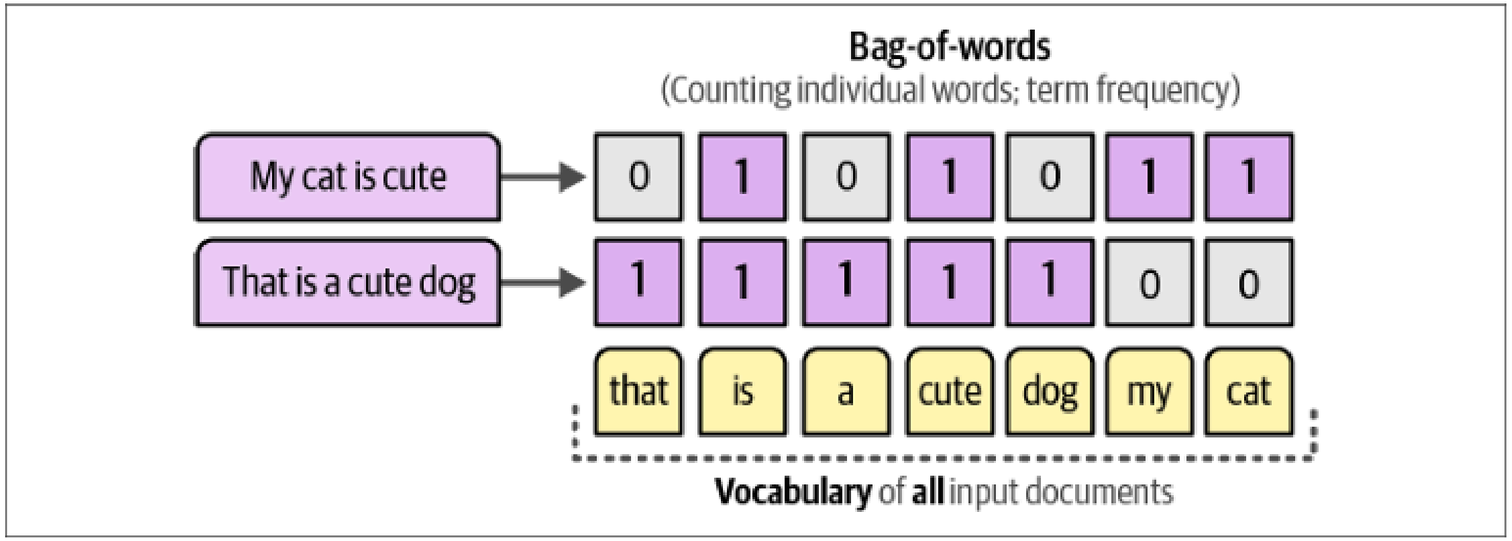


Figure 5-12. A bag-of-words counts the number of times each word appears inside a document.

- There are two caveats, however. First, this is a representation on a document level and we are interested in a cluster-level perspective. To address this, the frequency of words is calculated within the entire cluster instead of only the document, as illustrated in Figure 5-13

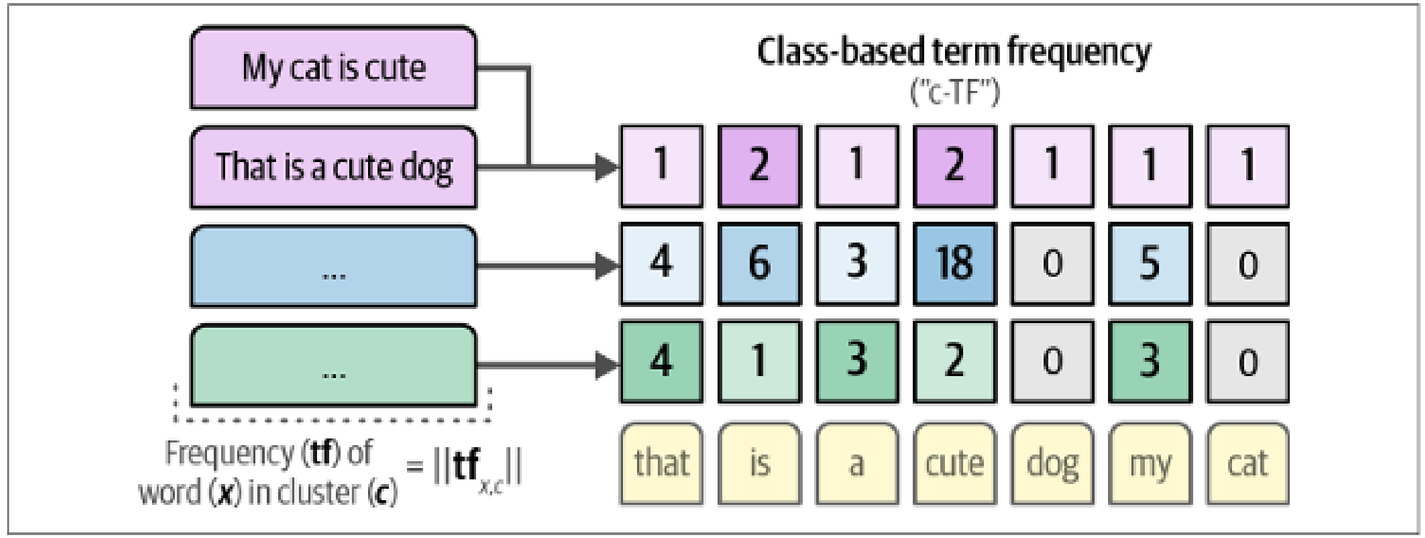


Figure 5-13. Generating c-TF by counting the frequency of words per cluster instead of per document.

Course 1
Large Language Models (LLMs) & Agentic AI



Completed: 50% / 100%

Academia.ai © 2026 | Text Clustering and Topic Modeling